Recommendation Systems Mini Project

Build it end to end. Prove you understand it. Put it on your CV.

1. What this project is for

This is not about marks. It has two goals:

- Understand every stage of a recommender pipeline by building one yourself, from raw data to a working demo.
- Finish with a project you can put on your CV and defend in an interview.

You have one week, and you may use AI tools. That changes what "good" means. A basic recommender can now be generated in an hour, so a project that only runs is not worth much. What is worth something is a project where you can show you understood it, tested it honestly, and know where it breaks.

- Teams: pairs (66 students, 32 teams) fexlible.
- Topics: choose one from the menu. At most 2 teams per topic(flexible). Teams sharing a topic must choose different twists (Section 9).
- Own topic: allowed with approval. It needs real user-item interaction data (ratings, clicks, purchases or plays) from a public source.
- Start from the starter notebook: Week10_Project_Starter_Template.ipynb.
- Dates: proposal _____ | mid-point check _____ | final submission and walkthrough _____ (instructor fills in).

Company names in this handout show where a problem appears. They do not mean the company uses these datasets or methods.

2. Using AI well

AI tools are allowed and expected. The rules are about ownership, not about banning tools:

1. You must be able to explain every line you submit, and re-derive one prediction by hand (Section 4).
2. Every number in your README and CV comes from running your own notebook. Never paste a number an AI told you.
3. Log what you used AI for and what you checked yourself (the AI-use log).

Prompts that make you learn, not just copy

Instead of	Try
"Write a recommender for MovieLens."	"Explain user-based CF step by step, with a 3-user example, then show me the code."
"Fix this error."	"Why does this error happen? Then show the fix and tell me how to avoid it next time."
Accepting the first answer.	"List every way this pipeline could leak test data into training, and point to the lines."
"Is my project good?"	"Act as a sceptical interviewer. Ask me 5 hard questions about this notebook."

Known AI traps: check your code for each one

- Fitting TF-IDF, a scaler, means or similarities on all the data, including the test set (leakage).
- Treating unrated cells as a rating of zero instead of "no opinion".
- Using a random split on data that has timestamps, which flatters the model.

- Recommending items the user has already rated or bought.
- Computing a metric on training data, or comparing models on different test sets.
- Reporting a result without baselines (global mean, user mean, most popular).
- Great accuracy that comes only from recommending popular items. Check coverage.
- Made-up column names, file formats or library functions. Always look at the real data and run the code.
- README claims that are not in the notebook.

3. What you must build (the pipeline)

Each stage is a titled section of your notebook, in this order.

1. Problem and users: who gets the recommendation and what decision it supports.
2. Data: source, licence, size, cleaning steps, sparsity number.
3. EDA: rating or interaction distribution, popularity long tail, at least two plots.
4. Baselines: global mean, user mean, most popular.
5. Content-based model: item features, cosine similarity, user profile.
6. Collaborative model: user-based or item-based, mean-centred (explicit) or confidence-weighted (implicit).
7. Matrix factorization: a simple SVD, compared against your neighbourhood CF. The starter notebook gives you svd_scores.
8. Honest evaluation: split first, fit on train only, time-based split if you have timestamps. RMSE (explicit data) plus precision@k, recall@k and NDCG@k. One results table with baselines.
9. Cold start or hybrid: show how a new item or user is handled, or a tuned blend of two models.
10. Demo: a function recommend(user_or_item, n) that prints readable results (titles, not IDs). Optional upgrade: a Streamlit app.

The metrics listed on each project card are the minimum for that topic. Add ranking metrics wherever the card lists only RMSE.

4. The understanding pack (what shows it is yours)

This is the part AI cannot do for you. It goes in Sections 11 to 14 of the starter notebook and in your README.

Item	What to do
A. By-hand check	Pick one prediction from your collaborative model. Write the formula with the real numbers, compute it on paper, and show your code gives the same value (up to rounding). You practised this in the lab.
B. Deliberate-leak experiment	On purpose, fit your similarities or means on train plus test and measure how much the metric improves. Report honest versus leaky numbers. This proves you know what leakage looks like.
C. Decision log	At least 5 decisions (for example the value of k, the split, the filter threshold). For each: alternatives tried, a number as evidence, and your choice.
D. Bug log	At least 2 real problems you hit and fixed, with before and after numbers.
E. Break-it tests	Three tests: a brand-new user, a brand-new item, and a user with one rating (or all 5-star ratings). Say what happened and what you did.
F. AI-use log	What you asked AI to do, what you checked or changed, and one thing it got wrong.
G. Walkthrough	5 minutes. Each partner explains a different stage, then answers two "what if" questions (see Section 11).

Also write two sentences in your README: what you would do with 100 times more data, and how industry systems extend your approach. These become your interview answers.

5. One-week plan

Day	Build	Understanding pack
1	Finish the lab notebook. Pick a topic. Download the data and confirm it loads. Submit the proposal form (Appendix A).	Decide your split strategy.
2	Clean the data, EDA, baselines.	Start the decision log.
3	Content-based model. Mid-point check.	Note any bugs as they happen.
4	Collaborative model and SVD.	By-hand check.
5	Evaluation table, cold start or hybrid.	Deliberate-leak experiment.
6	Demo function (and Streamlit if time). README and CV bullet.	Break-it tests, AI-use log.
7	Re-run the notebook top to bottom on a fresh Colab. Final push to GitHub.	Rehearse the walkthrough.

6. Choose a topic

The level on each project describes how hard the data is to work with and the kind of CV signal it gives. The pipeline and the understanding pack are the same for everyone. Pick the one you can finish well in a week.

Level	Data and effort	CV signal
Foundation	Moderate-size explicit ratings that load easily.	You can build and honestly evaluate a recommender.
Intermediate	Implicit feedback, messy data, text features or a cold-start design.	You can handle real data and justify choices with evidence.
Advanced	Large data, two-stage design, scaling or location context. Expect sampling and extra setup.	You can work towards production constraints.

ID	Topic	Level	Data type	Core concept
P01	Movie recommender (MovieLens)	Foundation	Explicit	Mean-centring, k
P02	Book recommender (Goodbooks-10k)	Foundation	Explicit	Popularity bias
P03	Joke recommender (Jester): dense vs sparse	Foundation	Explicit	Density vs sparsity
P04	Anime recommender (MyAnimeList)	Foundation	Explicit+implicit	Explicit vs implicit
P05	Recipe recommender with dietary constraints (Food.com)	Foundation	Explicit	Filter, then rank
P06	Book-Crossing: cleaning messy real data	Intermediate	Explicit+implicit	Data cleaning, k-core
P07	Hybrid movie recommender with cold-start test (MovieLens + TMDB)	Intermediate	Explicit+text	Hybrid, cold start

P08	News recommender (MIND)	Intermediate	Clicks	Recency, time split
P09	Music recommender from listening history (Last.fm)	Intermediate	Play counts	Implicit confidence
P10	Grocery "buy again" and "you may also like" (Instacart)	Intermediate	Purchases	Repeat vs discovery
P11	E-commerce product recommender with review text (Amazon Reviews 2023)	Intermediate	Explicit+text	Random vs time split
P12	Game recommender (Steam)	Intermediate	Implicit	Hours as confidence
P13	Stay recommender when collaborative filtering fails (Inside Airbnb)	Intermediate	Reviews	When CF fails
P14	Research-paper recommender with cold-start articles (CiteULike)	Intermediate	Libraries+text	Cold-start items
P15	Local business recommender with location context (Yelp)	Advanced	Explicit+geo	Context, segments
P16	Two-stage fashion recommender (H&M)	Advanced	Purchases	Two-stage, MAP@12
P17	Session-based recommender for anonymous visitors (Retailrocket)	Advanced	Sessions	Session-based
P18	Scaling item-item search with approximate nearest neighbours (MovieLens full)	Advanced	Explicit (33M)	Scale, ANN
P19	Fairness and bias audit of a recommender (MovieLens 1M)	Intermediate	Explicit+groups	Group fairness
P20	Explainable recommender ("because you liked...")	Intermediate	Explicit+text	Explanations

7. Project cards

Each card lists the industry link, the data, the one concept your project must demonstrate, pipeline notes, and a CV bullet template. Replace every [X] with a number you measured. If you cannot measure it, delete that part of the bullet.

P01 Movie recommender (MovieLens)

Level (data difficulty)	**Foundation**
Industry link	OTT platforms (Netflix, Prime Video, Hotstar): rating prediction and "more like this" rows.
Dataset	MovieLens latest-small: 100,000 ratings, 600 users, 9,000 movies, with genres, free-text tags and TMDB/IMDb links. grouplens.org/datasets/movielens/latest
Concept to demonstrate	Mean-centring and neighbourhood size. Show why a centred similarity beats a raw one, and how k changes the error.
Pipeline notes	Content: genres + tags with TF-IDF. Collaborative: item-based and user-based with centred cosine. Baselines: global, user and item mean. Metrics: RMSE, precision@5. Demo: recommend(user_id, n) with titles.
CV bullet (template)	*Built an end-to-end movie recommender on MovieLens (100k ratings, 600 users) combining TF-IDF content filtering with mean-centred item-based collaborative filtering; reduced RMSE by [X]% over a user-mean baseline using a leakage-free rating split.*

P02 Book recommender (Goodbooks-10k)

Level (data difficulty)	**Foundation**
Industry link	Book retail and reading apps (Goodreads, Kindle, Amazon Books).
Dataset	Goodbooks-10k: 10,000 books, 53,424 users, about 6 million ratings (1 to 5), plus book tags and to-read lists. github.com/zygmuntz/goodbooks-10k
Concept to demonstrate	Popularity bias and the long tail. Measure what share of your recommendations come from the 100 most-rated books, and the share of the catalogue ever recommended (coverage).
Pipeline notes	Content: book tags (book_tags.csv) + title/author TF-IDF. Collaborative: item-based on a sparse matrix. Keep Colab-sized: users with at least 20 ratings, sample 10,000 users. Metrics: RMSE, precision@10, coverage.
CV bullet (template)	*Built a hybrid book recommender over Goodbooks-10k (6M ratings) using tag-based TF-IDF and item-based CF on sparse matrices; quantified popularity bias (top-100 titles made up [X]% of recommendations) and catalogue coverage of [Y]%.*

P03 Joke recommender (Jester): dense vs sparse

Level (data difficulty)	**Foundation**
Industry link	Onboarding questionnaires on streaming and shopping apps that ask new users to rate or pick a few favourites to avoid a cold start.
Dataset	Jester dataset 1: about 4.1 million ratings of 100 jokes by 73,421 users, on a -10 to +10 scale (99 means not rated). Free for research with citation. eigentaste.berkeley.edu/dataset

Concept to demonstrate	How data density controls collaborative filtering quality. Jester is unusually dense, so you can hide ratings at random (10%, 30%, 50%, 70%, 90% hidden) and plot RMSE against sparsity.
Pipeline notes	Collaborative: user-based and item-based with centred cosine. Content: use joke text if the dataset page provides it (check before committing); otherwise state clearly that content features are derived from the rating matrix. Metrics: RMSE vs sparsity curve, precision@5. Compare with a gauge-set approach: ask a new user to rate only 5 jokes.
CV bullet (template)	*Analysed collaborative filtering on the Jester joke dataset (4.1M ratings); measured how RMSE degrades from [X] to [Y] as sparsity rises from 10% to 90% and designed a 5-item onboarding set that recovers [Z]% of full-profile accuracy.*

P04 Anime recommender (MyAnimeList)

Level (data difficulty)	**Foundation**
Industry link	Streaming catalogues with strong genre structure (Crunchyroll, Netflix, Hotstar).
Dataset	MyAnimeList database (Hernan4444; also on Kaggle as Anime Recommendation Database 2020): anime metadata (genres, synopsis, scores) and user lists. github.com/Hernan4444/MyAnimeList-Database
Concept to demonstrate	Explicit and implicit signals in one dataset. Some list entries are watched without a rating, or only planned. Read the data dictionary, decide how to treat each case, and show the effect of your choice.
Pipeline notes	Content: genres + synopsis TF-IDF. Collaborative: item-based. Metrics: RMSE on rated entries, precision@k with "watched" as a weaker positive. Sample users to stay Colab-sized.
CV bullet (template)	*Developed an anime recommender on MyAnimeList data combining synopsis/genre TF-IDF with item-based CF; evaluated implicit (watched) versus explicit (rated) signals and improved precision@10 from [X] to [Y] over a popularity baseline.*

P05 Recipe recommender with dietary constraints (Food.com)

Level (data difficulty)	**Foundation**
Industry link	Food delivery and meal kits (Swiggy, Zomato, HelloFresh): recommendations must respect diet and allergy rules before taste.
Dataset	Food.com Recipes and Interactions (Kaggle): on the order of 200k recipes and 1M user-recipe interactions with ratings, ingredients, tags and nutrition. Search the exact name on Kaggle and read the data card.
Concept to demonstrate	Filter, then rank. Apply hard constraints (vegetarian, no nuts, calorie cap) first, then score. Show that constraint satisfaction is 100% and measure what it costs in accuracy.
Pipeline notes	Content: ingredients + tags TF-IDF. Collaborative: item-based on users with at least 10 interactions. Metrics: RMSE, precision@10, constraint-violation rate (must be 0). Demo: recommend(user, n, diet=...).
CV bullet (template)	*Built a constraint-aware recipe recommender on Food.com data (about 1M interactions) with ingredient-based TF-IDF and item-based CF; enforced dietary filters with 0% violations at a cost of only [X] points of precision@10.*

P06 Book-Crossing: cleaning messy real data

Level (data difficulty)	**Intermediate**
Industry link	E-book and library platforms (Kobo, Amazon, public-library apps) where data arrives dirty and very sparse.
Dataset	Book-Crossing (C.-N. Ziegler et al., Univ. of Freiburg; also on Kaggle as Book Recommendation Dataset): about 1.1 million ratings, 278k users, 271k books. Ratings are 1 to 10, and 0 means an implicit interaction.
Concept to demonstrate	Data cleaning decisions change results. Treat 0 correctly, fix invalid ages and duplicate ISBNs, apply k-core filtering, and show how the threshold (5, 10, 20 interactions) changes both accuracy and coverage.
Pipeline notes	Content: title, author, publisher TF-IDF. Collaborative: item-based on sparse matrices. Ranking metrics: precision@k, recall@k, NDCG@k. Ablation table across filtering thresholds.
CV bullet (template)	*Cleaned and modelled the Book-Crossing dataset (1.1M ratings, 99.9% sparse); showed that k-core filtering at [X] interactions lifted NDCG@10 from [Y] to [Z] while keeping [W]% of the catalogue.*

P07 Hybrid movie recommender with cold-start test (MovieLens + TMDB)

Level (data difficulty)	**Intermediate**
Industry link	OTT launches of new titles, which have no ratings on day one.
Dataset	MovieLens ratings and links.csv (which carries tmdbId) joined to the TMDB 5000 Movie Dataset on Kaggle (overview, keywords, cast, crew). Restrict to the overlap. grouplens.org/datasets/movielens/latest
Concept to demonstrate	Hybrid design and ablation. Hide all ratings of 10% of the movies and test which model can still score them.
Pipeline notes	Content: overview, keywords, cast, director as one TF-IDF text field. Collaborative: item-based. Hybrid: weighted blend with alpha tuned on a validation split. Report CF-only, CB-only and hybrid on warm and cold items separately.
CV bullet (template)	*Designed a hybrid recommender (TF-IDF over TMDB metadata + item-based CF on MovieLens); on cold-start movies the hybrid reached RMSE [X] versus [Y] for CF-only, which cannot score unseen items.*

P08 News recommender (MIND)

Level (data difficulty)	**Intermediate**
Industry link	News apps (Inshorts, Dailyhunt, Google News, MSN) where articles expire within hours.
Dataset	MIND, the Microsoft News dataset: click impressions, news titles, abstracts and categories. Use the small split. Microsoft Learn: MIND / Microsoft News dataset
Concept to demonstrate	Implicit clicks and short item lifetimes. Every article is a cold-start item, so content and recency carry the system. Use a time-aware split: train on earlier days, test on later ones.
Pipeline notes	Content: title + abstract TF-IDF, category. User profile: average of clicked articles. Collaborative: item co-click similarity. Recency boost. MIND's usual metrics are AUC, MRR and nDCG@5/10.

CV bullet (template)	*Built a news recommender on the MIND dataset using TF-IDF user profiles and a recency boost; with a time-based split, reached nDCG@10 of [X] versus [Y] for a most-popular baseline.*

P09 Music recommender from listening history (Last.fm)

Level (data difficulty)	**Intermediate**
Industry link	Music streaming (JioSaavn, Spotify, Gaana, YouTube Music).
Dataset	Last.fm datasets from Oscar Celma: LASTFM-360K (about 359k users, about 17.5M user-artist play-count rows) and LASTFM-1K (992 users, about 19M timestamped listens). Non-commercial licence, fine for coursework; say so in your README. zenodo.org/records/6090214
Concept to demonstrate	Implicit feedback. Play counts are not ratings. Transform them (log, capped), build confidence weights, and show why raw counts mislead.
Pipeline notes	Item-item CF over artists on a sparse user-artist matrix. Content: artist tags if you add them. Leave-last-out evaluation per user with recall@k and NDCG@k. Compare raw counts, log counts and binary.
CV bullet (template)	*Built an implicit-feedback music recommender on Last.fm data (up to 17.5M play-count records) using log-scaled confidence and item-item CF on sparse matrices; reached recall@20 of [X] versus [Y] for a popularity baseline.*

P10 Grocery "buy again" and "you may also like" (Instacart)

Level (data difficulty)	**Intermediate**
Industry link	Quick commerce and grocery apps (BigBasket, Blinkit, Swiggy Instamart).
Dataset	Instacart Market Basket Analysis (Kaggle competition data): orders, products, aisles, departments. A Kaggle account and acceptance of the competition rules are needed. Search the exact name on Kaggle.
Concept to demonstrate	Repeat purchase versus discovery. Split recommendations into "buy it again" (items the user bought before) and "new for you" (association-based), and show they need different models.
Pipeline notes	Item-item co-occurrence with lift or cosine on baskets. Content: aisle and department. Evaluation: hold out each user's last order, report precision@10 and recall@10 for both lists separately.
CV bullet (template)	*Built a grocery recommender on Instacart orders using repeat-purchase scoring and item-item co-occurrence; held out each user's last basket and achieved recall@10 of [X], a [Y]% lift over a popularity baseline.*

P11 E-commerce product recommender with review text (Amazon Reviews 2023)

Level (data difficulty)	**Intermediate**
Industry link	Marketplaces (Amazon, Flipkart, Meesho).
Dataset	Amazon Reviews 2023 by the McAuley Lab on Hugging Face: reviews and item metadata by category. Pick one small category (for example Magazine_Subscriptions or All_Beauty). huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023
Concept to demonstrate	k-core filtering and a time-based split. Show how filtering changes the data and why a random split overstates quality when the real use is predicting the future.

Pipeline notes	Content: review text and product title TF-IDF. Collaborative: item-based. Compare random split versus temporal split in one table. Metrics: RMSE, NDCG@10.
CV bullet (template)	*Built a product recommender on Amazon Reviews 2023 using review-text TF-IDF and item-based CF; showed a random split overstated NDCG@10 by [X]% compared with a time-based split.*

P12 Game recommender (Steam)

Level (data difficulty)	**Intermediate**
Industry link	Game stores (Steam, Epic, Google Play Games).
Dataset	Game Recommendations on Steam (Kaggle, by Anton Kozyriev): user reviews with a recommended yes/no flag and hours played, plus game metadata. Search the exact name on Kaggle. Sample users to stay Colab-sized.
Concept to demonstrate	Implicit feedback with a binary label. Use hours played as a confidence weight and compare against the recommended flag.
Pipeline notes	Content: game tags and descriptions TF-IDF. Collaborative: item-item on a sparse matrix. Metrics: precision@k, recall@k, NDCG@k. Ablation: binary versus hours-weighted.
CV bullet (template)	*Built a Steam game recommender using hours-played confidence weighting and tag-based TF-IDF; improved NDCG@10 from [X] to [Y] over a binary-feedback baseline.*

P13 Stay recommender when collaborative filtering fails (Inside Airbnb)

Level (data difficulty)	**Intermediate**
Industry link	Travel and stays (Airbnb, OYO, MakeMyTrip).
Dataset	Inside Airbnb (insideairbnb.com, get the data): per-city listings with amenities, price, neighbourhood and descriptions, and reviews that give (reviewer, listing) pairs. Pick one city.
Concept to demonstrate	When CF fails. Most reviewers review one listing in a city. Measure the sparsity, show CF's weak results, then build a content and constraint-based recommender (budget, neighbourhood, room type) and be honest about how you evaluate it.
Pipeline notes	Content: description and amenities TF-IDF, price band, neighbourhood. CF: item-item on co-reviewed listings. Evaluation proxy: held-out reviewed listing appears in top-k. Constraint filters on price and area.
CV bullet (template)	*Built a constraint-aware stay recommender on Inside Airbnb data; demonstrated that CF reached only [X] recall@10 at [Y]% sparsity, and a content-plus-constraints model raised it to [Z].*

P14 Research-paper recommender with cold-start articles (CiteULike)

Level (data difficulty)	**Intermediate**
Industry link	Scholarly tools (Semantic Scholar, Mendeley) and any catalogue where new items keep arriving.
Dataset	CiteULike (citeulike-a or citeulike-t), used in the Collaborative Topic Regression paper by Wang and Blei (2011): users' article libraries with titles and abstracts. Hosting varies, so confirm you can download it before choosing this topic.
Concept to demonstrate	Cold-start items. Hold out whole articles (never seen in training) and show that CF cannot score them while text-based methods can.

Pipeline notes	Content: abstract TF-IDF. Collaborative: item-based. Hybrid on warm items, content-only fallback on cold items. Report warm and cold results separately with recall@k.
CV bullet (template)	*Built a paper recommender on CiteULike combining abstract TF-IDF with collaborative filtering; on held-out cold-start articles, recall@20 was [X] versus 0 for CF-only.*

P15 Local business recommender with location context (Yelp)

Level (data difficulty)	**Advanced**
Industry link	Restaurant and local discovery (Zomato, Swiggy Dineout, Google Maps).
Dataset	Yelp Open Dataset: businesses, reviews, users, check-ins as large JSON files. Free for academic use under Yelp's terms, so read them first. Pick one metro area.
Concept to demonstrate	Context-aware recommendation at scale. Combine collaborative scores with category and location filters, process large files in chunks, and evaluate separately for heavy and light users.
Pipeline notes	Stream and filter JSON to one metro. Content: categories, attributes, review text. CF: item-item on sparse matrices after k-core. Geographic filter. Report metrics by user-activity segment. Deploy a Streamlit demo.
CV bullet (template)	*Built a location-aware restaurant recommender on the Yelp Open Dataset ([X]M reviews, one metro) with chunked processing, sparse item-item CF and category filters; reached NDCG@10 of [Y] and deployed it as a Streamlit app.*

P16 Two-stage fashion recommender (H&M)

Level (data difficulty)	**Advanced**
Industry link	Fashion e-commerce (Myntra, Ajio, Flipkart Fashion). Production systems use candidate generation followed by ranking.
Dataset	H&M Personalized Fashion Recommendations (Kaggle competition data): dated purchase transactions, customer and article metadata, images. Accept the competition rules. Sample about 10% of customers.
Concept to demonstrate	Candidate generation then ranking, with a time-based validation window (hold out the last week). Metric: MAP@12.
Pipeline notes	Candidates from recent popularity, repurchase and item-item similarity. Rank with a simple scoring model (weighted features or logistic regression). Content: article attributes. Report each candidate source's recall, then the final MAP@12. Streamlit demo.
CV bullet (template)	*Built a two-stage fashion recommender on H&M purchase data (candidate generation from three sources + a ranking model); reached MAP@12 of [X] on a held-out week, [Y]% above a popularity baseline.*

P17 Session-based recommender for anonymous visitors (Retailrocket)

Level (data difficulty)	**Advanced**
Industry link	E-commerce "customers who viewed this also viewed", for visitors who are not logged in.
Dataset	Retailrocket recommender system dataset (Kaggle): visitor events (view, addtocart, transaction) with item properties. Search the exact name on Kaggle.
Concept to demonstrate	No user profile. Build sessions from events, weight the funnel (view < add-to-cart < purchase), and predict the next item.

Pipeline notes	Co-visitation item-item with recency and funnel weights; content from item properties. Evaluate next-item hit-rate@k and MRR with a time-based split. Streamlit demo that takes a list of viewed items.
CV bullet (template)	*Built a session-based recommender on Retailrocket event logs using funnel-weighted co-visitation; achieved hit-rate@10 of [X] for next-item prediction, [Y]% above a popularity baseline.*

P18 Scaling item-item search with approximate nearest neighbours (MovieLens full)

Level (data difficulty)	**Advanced**
Industry link	Any catalogue with millions of items, where brute-force similarity is too slow.
Dataset	MovieLens latest (full): about 33M ratings, 86k movies, 331k users, a 335 MB download. grouplens.org/datasets/movielens/latest
Concept to demonstrate	Scalability. A dense 86k by 86k item similarity matrix does not fit in memory, so you must use sparse matrices, chunking and approximate search. Report neighbour recall versus latency and memory.
Pipeline notes	Sparse CSR matrices. Item embeddings with TruncatedSVD (a stretch beyond the syllabus, clearly labelled). Compare exact nearest neighbours (chunked) against FAISS or Annoy. Table: recall of the true top-10 neighbours, query latency, memory. Streamlit or FastAPI demo.
CV bullet (template)	*Scaled item-based CF to 33M ratings using sparse matrices and approximate nearest neighbours (FAISS/Annoy); achieved [X]% neighbour recall at [Y] ms per query, [Z] times faster than exact search.*

P19 Fairness and bias audit of a recommender (MovieLens 1M)

Level (data difficulty)	**Intermediate**
Industry link	Responsible AI. Platforms are increasingly asked to show how their recommenders behave for different groups of users.
Dataset	MovieLens 1M (about 1 million ratings, 6,000 users, 4,000 movies) from GroupLens, which includes gender, age group and occupation for users.
Concept to demonstrate	Aggregate metrics hide group differences. Compute RMSE and precision@k by gender and age group, check popularity bias by group, then try one mitigation (re-weighting or re-ranking) and report the trade-off. Use demographics only to audit, never to target.
Pipeline notes	Build one CF and one CB model, then a group-wise evaluation table with confidence intervals (bootstrap). Mitigation experiment. Written limitations section.
CV bullet (template)	*Audited a collaborative-filtering recommender on MovieLens 1M for group fairness; found an RMSE gap of [X] between age groups and reduced it to [Y] with re-weighting at a [Z]% cost in overall accuracy.*

P20 Explainable recommender ("because you liked...")

Level (data difficulty)	**Intermediate**
Industry link	"Because you watched" rows; explanations build user trust and help engineers debug.
Dataset	Reuse MovieLens latest-small or Goodbooks-10k. grouplens.org/datasets/movielens/latest

Concept to demonstrate	Explanations as a product feature. For each recommendation, show the evidence: top shared content terms, or the neighbours who rated it highly. Evaluate the explanations, not only the ranking.
Pipeline notes	Content-based with per-term TF-IDF contribution, plus neighbour-based evidence. Faithfulness test: remove the cited item from the profile and check whether the recommendation drops. Small user study (5 to 10 classmates, 1 to 5 helpfulness). Demo prints the reason with each item.
CV bullet (template)	*Built an explainable movie recommender that attaches a content or neighbour-based reason to every recommendation; a 10-person study rated explanations [X]/5 and a faithfulness test showed [Y]% of recommendations depend on the cited item.*

8. Check your data before you commit

- Download and load the dataset on day 1. Some need a Kaggle or Yelp account, and Kaggle competition data (Instacart, H&M) needs you to accept the competition rules.
- Keep it runnable on Colab: filter to active users and items, sample users, use scipy.sparse, and never build a dense users-by-items table for large data.
- If your dataset will not load by the end of day 1, switch topic. Do not spend the week on data access.

9. Twist menu (for teams sharing a topic)

If your topic is also taken by another team, pick a different twist and declare it in your proposal. Any team may add one for extra depth.

- Time-based split instead of random, with a table showing how much a random split overstates quality.
- Popularity-bias analysis and a re-ranking step that improves coverage.
- Explanations for every recommendation (as in P20).
- Segment analysis: results for light versus heavy users, or by user group.
- Cold-start simulation: hide all interactions of 10% of items or users.
- Scalability benchmark: time and memory against data size.
- A deployed demo with Streamlit or FastAPI.

10. Are you done? (checklist)

No marks. Tick every box before you submit. If a box is empty, the project is not finished.

Group	Tick when true
Pipeline	[ ] All 10 stages are present as titled sections.
Pipeline	[ ] The notebook runs top to bottom on a fresh Colab, with a requirements list.
Pipeline	[ ] Split happens before any fitting, and nothing is fitted on test data.
Pipeline	[ ] Baselines and all models are in one results table on the same test set.
Pipeline	[ ] The demo prints readable results and never recommends already-rated items.
Understanding pack	[ ] The by-hand check matches the code.
Understanding pack	[ ] The deliberate-leak result is reported (honest versus leaky).
Understanding pack	[ ] Decision log has 5 or more entries; bug log has 2 or more.
Understanding pack	[ ] Three break-it tests are done.
Understanding pack	[ ] The AI-use log is filled in.

Portfolio-ready	[ ] README has a one-line summary, results table, demo screenshot and limitations.
Portfolio-ready	[ ] CV bullet written with numbers from the notebook.
Portfolio-ready	[ ] GitHub repo is tidy and public.
Portfolio-ready	[ ] You can explain every stage without looking at the code.

11. Repo structure and walkthrough

Submit to GitHub under labs/week10_recsys/<team_name>/ containing:

- notebook.ipynb: the full pipeline, run top to bottom, with outputs saved.
- README.md: from the template at the end of the starter notebook.
- requirements.txt: libraries and versions.
- app.py: if you built a Streamlit or FastAPI demo.
- data/README.md: how to download the data. Do not commit large raw data.
- cv_bullet.txt: your final CV bullet.

The 5-minute walkthrough

Each partner explains a different stage of the pipeline in your own words, running the demo live. Then you answer two "what if" questions, for example:

- If I double k, what happens to your results and why?
- What happens to a brand-new item, and what would you change?
- Where could test data have leaked into training in your pipeline?
- Why does your best model beat the baseline, and how sure are you?

This is rehearsal for an interview. There is nothing to hide: explaining a mistake you caught is a strength.

12. Metrics cheat sheet

Metric	What it measures	Use when
RMSE	Typical size of the error in predicted ratings. Lower is better.	Explicit ratings.
Precision@k	Of the k items you recommended, what fraction the user actually liked.	Top-k lists.
Recall@k	Of all items the user liked in the test set, what fraction appear in your top k.	Implicit data, where liked items are few.
NDCG@k	Like precision, but a hit at rank 1 counts more than a hit at rank 10.	Ranking quality matters.
Coverage	Share of the catalogue that ever gets recommended.	Checking popularity bias.
Baselines	Global mean, user mean, item mean, most popular.	Always.

13. The map: where your project sits

Content-based and collaborative filtering are the foundation. Industry systems extend them. You build the first rows; the rest is reading, so you can say in an interview that you know where the field goes.

Approach	Idea	Your project

Popularity and rules	Recommend what is popular or frequently bought together.	Built (baseline)
Content-based	Match item features to a user profile.	Built
Collaborative filtering	Use similar users or similar items.	Built
Matrix factorization	Learn hidden taste vectors for users and items.	Built (simple SVD)
Hybrid	Blend several methods.	Built (cold start or hybrid)
Implicit-feedback models	Learn from clicks, plays and purchases with confidence weights (for example ALS, BPR).	Some topics
Two-tower embeddings	Neural networks learn user and item vectors; retrieval is nearest-neighbour search.	Read only
Sequence and transformer models	Use the order of a user's actions to predict the next item.	Read only
Generative recommendation	Generate item identifiers or whole pages the way language models generate text. An active research area.	Read only
LLM-assisted	Use language models for explanations, text understanding and cold start.	Read only
Bandits and reinforcement learning	Learn by trying recommendations and watching responses.	Read only

Large systems work in stages: retrieval (narrow millions of items to hundreds), ranking (score them carefully), re-ranking (rules, filters, diversity), then an online A/B test with real users. Your models are the retrieval and ranking ideas at small scale. An A/B test is what you could not do, and you should say so.

14. CV guide

Bullet formula

Action verb + what you built + data size + method + result against a baseline + how you evaluated + link. One to two lines.

Level	Example shape (replace with your own numbers)
Foundation	Built a movie recommender on MovieLens (100k ratings) with TF-IDF content filtering, mean-centred item-based CF and SVD; cut RMSE by [X]% versus a user-mean baseline on a leakage-free split, and showed leakage would have inflated it by [Y]%. GitHub: <link>
Intermediate	Developed an implicit-feedback music recommender on Last.fm ([X]M plays) with log-scaled confidence and item-item CF; reached recall@20 of [Y] versus [Z] for a popularity baseline using a leave-last-out split. GitHub: <link>
Advanced	Built a two-stage recommender on H&M purchase data (candidate generation + ranking), validated on a held-out week with MAP@12 of [X]; deployed as a Streamlit app. GitHub: <link> | Demo: <link>

Rules

- Use only numbers from your own notebook. An invented number can end an interview.
- Say it is an academic project. Do not write "deployed to production" or "increased revenue".

- Mention how you evaluated (time-based split, baselines, leakage check). Interviewers care about this more than the library names.
- Always include the GitHub link, and make sure the README matches the bullet.
- Skills you can honestly list: Python, pandas, NumPy, scikit-learn, TF-IDF, cosine similarity, collaborative filtering, matrix factorization (SVD), evaluation (RMSE, precision@k, NDCG), SciPy sparse matrices, Streamlit or FastAPI (if you built the demo).
- If you used AI tools, say so if asked, and be ready to explain every part.

15. Dataset notes

- Check licences and say which one in your README. The Last.fm data is licensed for non-commercial use, Jester asks you to cite its paper, and Yelp has its own terms for the open dataset, so read them first.
- MovieLens' own page says its datasets are not appropriate for reporting research results. That is fine for a course project, but do not present your numbers as research findings.
- Links were checked in October 2026 where a link is given. For datasets without a link, search the exact name; hosting changes over time.

Dataset pages used for this handout:

- MovieLens latest: grouplens.org/datasets/movielens/latest
- Goodbooks-10k: github.com/zygmuntz/goodbooks-10k
- Jester: eigentaste.berkeley.edu/dataset
- MyAnimeList database: github.com/Hernan4444/MyAnimeList-Database
- MIND news dataset: learn.microsoft.com/azure/open-datasets/dataset-microsoft-news
- Last.fm (Celma): zenodo.org/records/6090214
- Amazon Reviews 2023: huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023
- UCI Online Retail II (an extra option for a transaction-based topic): archive.ics.uci.edu/dataset/502/online+retail+ii

Appendix A: Project proposal form (submit on day 1)

Field	Your answer
Team name	
Member 1 (name, roll number)	
Member 2 (name, roll number)	
Topic ID and name	
Level	
Twist (required if the topic is shared)	
Dataset name and link	
Dataset downloaded and loads in Colab?	Yes / No
Number of users, items, interactions	
Do you have timestamps?	Yes / No
One sentence: who is the user and what is recommended?	
Which AI tools do you plan to use?	

Instructor: approved ____ needs change ____ Notes: _____

Appendix B: Interview Q&A on recommenders

Short model answers. Practise saying each in your own words, using your project as the example.

Question	Short model answer
What is the difference between content-based and collaborative filtering?	Content-based recommends items similar to what the user liked, using item features. Collaborative uses the behaviour of similar users or items and needs no item features, but needs enough overlapping interactions.
Why did you mean-centre the ratings?	Users use the scale differently. Centring removes each user's bias so similarity reflects taste. Without it, similarity is always positive and cannot show opposite tastes.
What is the cold-start problem and how did you handle it?	A new user or item has no history, so collaborative methods cannot score it. Content features, popularity fallbacks, onboarding questions or a hybrid help. Name what you did in your project.
How did you split the data, and why?	Split ratings before fitting anything, and use a time-based split when timestamps exist, because the real task is predicting the future. Fit means, similarities and vocabularies on train only.
What is data leakage? Show an example.	Information from the test set influencing training. Example: computing user similarities from the full matrix. Your deliberate-leak experiment is the evidence, with the metric before and after.

Why compare with baselines?	A model that cannot beat the global mean, user mean or most popular is not learning anything useful. Baselines also show whether the improvement is worth the complexity.
Explicit versus implicit feedback?	Explicit is a rating the user gave. Implicit is behaviour such as clicks or plays, which is noisy and has no negatives. Implicit data needs confidence weights and ranking metrics.
Why cosine similarity? What are the alternatives?	It compares direction and ignores magnitude, which suits sparse vectors. Alternatives: Pearson correlation, Jaccard similarity for sets, Euclidean distance. Be ready to say what happens with an all-zero vector.
How did you choose k, the number of neighbours?	Try several values on a validation split and look at the curve. Very small k is noisy, very large k blurs differences. Cite your numbers.
What is sparsity and why does it matter?	The share of empty cells in the user-item matrix. High sparsity means few co-rated items, so neighbourhood similarity becomes unreliable.
Why use ranking metrics and not only RMSE?	Users see a ranked list, not predicted ratings. A model can have low RMSE and still rank poorly. NDCG gives more credit to hits near the top.
What is popularity bias?	Recommending mostly popular items, which can score well while ignoring the long tail and hurting coverage. Check coverage, and consider re-ranking.
User-based or item-based CF: which scales better?	Item-based is often more stable and can be precomputed when there are more users than items. State the trade-offs and what held in your data.
What is matrix factorization and how does it differ from neighbourhood CF?	It learns k hidden factors per user and item whose product approximates the rating, instead of comparing raw rating vectors. It handles sparsity better and is the bridge to embedding models.
How would you scale this to 100 times more data?	Sparse matrices, sampling, candidate generation then ranking, approximate nearest-neighbour search, precomputing item similarities, and incremental updates.
How would you evaluate it in production?	Offline metrics first, then an online A/B test measuring click-through, watch time or purchases, with guardrails such as diversity and latency. Say plainly that you only did the offline part.
How do industry systems extend what you built?	They use stages (retrieval, ranking, re-ranking), learned embeddings such as two-tower models, sequence models, and implicit feedback at scale. Your project is the foundation of these ideas.
What did AI get wrong in your project, and how did you catch it?	Use your AI-use log. A specific, honest example (a leaky split, a made-up function, a wrong formula) is the best answer you can give.